

ClanTree Plugin

Performance & Code Quality Analysis
Generated 20 June 2026

Executive Summary

ClanTree v1.0.0 is a central integration plugin managing personal and clan perk upgrades across currency, shop, and clan plugins. Its position as a hub means any stall it introduces on the Rust main thread propagates across every dependent system. This audit identified **3 Critical**, **5 High**, and **5 Medium** findings. The most severe is that all player and clan data is loaded and saved synchronously on the main thread using **File.ReadAllText / File.WriteAllText + JsonConvert**, with no background I/O worker. On a high-population server this directly produces the 100 ms stalls observed during peak raid windows.

ID	Severity	Area	Title
CT-01	CRITICAL	Persistence	Synchronous File.ReadAllText on player connect
CT-02	CRITICAL	Persistence	Coroutine HandleSaves() still writes JSON on main thread
CT-03	CRITICAL	Persistence	SaveClanData() called synchronously on player disconnect
CT-04	HIGH	Bug	RemoveOldClans() calls Directory.Delete() on a file path
CT-05	HIGH	Persistence	JSON format vs protobuf binary — significant speed deficit
CT-06	HIGH	Integration	IsClanAuthenticated() deserialises a JObject on every skill purchase
CT-07	HIGH	UI	Full CUI rebuild + CuiHelper.AddUi() on every menu open
CT-08	HIGH	Memory	GetClanMembers() returns pooled List — Pool leak risk
CT-09	MEDIUM	Performance	ShouldRemoveClanData() O(n) scan on every player disconnect
CT-10	MEDIUM	Correctness	TOCTOU race in currency: balance check then separate withdraw
CT-11	MEDIUM	Thread-safe	Instance bool flags IsInstantGathering / Deforesting
CT-12	MEDIUM	Thread-safe	FindEntitiesOfType uses shared Vis.colBuffer
CT-13	MEDIUM	Performance	SubscriptionInfo.subscribers uses List linear-scan Remove

Critical Findings

CT-01 CRITICAL Synchronous File.ReadAllText on player connect

On every player connect SetupPlayer() calls LoadPlayerData() which executes a synchronous **File.ReadAllText** followed by **JsonConvert.DeserializeObject** on the main thread. During a raid peak when many players reconnect simultaneously, each call blocks the main thread for the full disk read and JSON parse time — easily 5-30 ms each — causing cascading stall spikes.

```
// Line 697 - LoadPlayerData() - BLOCKS MAIN THREAD
private PlayerInfo LoadPlayerData(ulong id)
{
    var path = $"{FileDirectory}{id}.json";
    if (!File.Exists(path)) return new PlayerInfo();
    var json = File.ReadAllText(path); // ← blocking syscall
    return JsonConvert.DeserializeObject<PlayerInfo>(json); // ← GC pressure
}
```

Fix: Cache all player data at server startup into a dictionary during OnServerInitialized using a background **Thread** (not a coroutine). On connect, do a dictionary lookup — zero I/O. For players whose data is not yet loaded, queue an async read exactly as Kits v2 does with its PlayerSaveWorker and enqueue the player for delayed setup.

```
// Target pattern (Kits v2 style)
// Startup: Thread reads all .json files off main thread into _cache
// OnPlayerConnected: _cache.TryGetValue(player.userID, out var data)
// → hit: use immediately → miss: enqueue background read
```

CT-02**CRITICAL****Coroutine HandleSaves() still writes JSON on main thread**

HandleSaves() (line 1365) is a Unity coroutine — it yields between batches of 10 writes using yield return null, but each iteration still calls **JsonConvert.SerializeObject + File.WriteAllText on the main thread**. A coroutine does not move work to a background thread; it merely spreads it across multiple frames. With 100+ online players, each write still stalls the frame it executes on. The save interval is typically triggered by OnServerSave every 5 minutes, producing a 10-frame stall cluster.

```
// Line 1375 - still on main thread despite yield
private IEnumerator HandleSaves(bool saveOnUnload)
{
    int batchSize = 10;
    for (int i = 0; i < players.Count; i += batchSize)
    {
        foreach (var id in players.GetRange(i, ...))
        {
            var json = JsonConvert.SerializeObject(data); // ← main thread
            File.WriteAllText(path, json); // ← main thread
        }
        yield return null; // spreads across frames, does NOT offload I/O
    }
}
```

Fix: Introduce a dedicated background Thread with an AutoResetEvent queue (identical to the PlayerSaveWorker pattern in Kits v2). Serialise to byte[] on the main thread, then enqueue the byte array. The worker thread calls File.WriteAllBytes. Add a dirty flag and jitter delay to avoid write storms.

```
// Target worker pattern
private Thread _saveWorker;
private AutoResetEvent _saveSignal = new AutoResetEvent(false);
private ConcurrentQueue<(string path, byte[] data)> _saveQueue = new();

void EnqueueSave(string path, PlayerInfo info)
{
    var bytes = Encoding.UTF8.GetBytes(JsonConvert.SerializeObject(info));
    _saveQueue.Enqueue((path, bytes));
    _saveSignal.Set();
}

void SaveWorkerLoop()
{
    while (_running)
    {
        _saveSignal.WaitOne();
        while (_saveQueue.TryDequeue(out var item))
        {
            File.WriteAllBytes(item.path, item.data);
        }
    }
}
```

CT-03**CRITICAL****SaveClanData() called synchronously on player disconnect**

When a player leaves or a clan disbands, `HandlePlayerGone()` → `ShouldRemoveClanData()` → `SaveClanData()` is called **synchronously outside any coroutine** (lines 1951, 769). This runs `JsonConvert.SerializeObject` + `File.WriteAllText` directly on the `OnPlayerDisconnected` hook — a main-thread event. In a raid scenario with players cycling in/out frequently, this fires multiple times per second.

```
// Line 1951 - OnClanMemberGone path
void HandlePlayerGone(ulong id, List<ulong> members, string tag, bool disbanded)
{
    ...
    if (!ShouldRemoveClanData(tag)) return;
    SaveClanData(tag, null, true); // ← SYNCHRONOUS FILE WRITE on disconnect
}

// Line 769 - SaveClanData
void SaveClanData(string tag, ClanInfo data, bool remove)
{
    var json = JsonConvert.SerializeObject(data);
    File.WriteAllText(path, json); // ← blocks caller (main thread)
}
```

Fix: Route all clan saves through the same background worker queue as CT-02. Mark the `ClanInfo.RequiresSave` flag on disconnect and let the periodic save cycle drain the dirty set. Only perform immediate synchronous saves in `Unload()` where blocking is acceptable.

High Severity Findings

CT-04

HIGH

RemoveOldClans() calls Directory.Delete() on a file path (BUG)

RemoveOldClans() (line 1317) calls Directory.Delete(file) where file is a **file path string** obtained from Directory.GetFiles(). Directory.Delete on a regular file throws an IOException. This means **old clan data files are never actually cleaned up**. The exception is silently swallowed inside the coroutine, so the bug goes unnoticed but disk usage grows unboundedly across wipes.

```
// Line 1317-1338 - BUG: Directory.Delete on a file path
private IEnumerator RemoveOldClans()
{
    foreach (string file in Directory.GetFiles(ClanDirectory))
    {
        ...
        Directory.Delete(file); // ← WRONG: throws IOException
        // should be: File.Delete(file)
    }
}
```

Fix: Replace Directory.Delete(file) with File.Delete(file). One character change — but critical for disk hygiene across wipes.

```
File.Delete(file); // correct
```

CT-05

HIGH

JSON serialisation — 3-8× slower than binary protobuf

ClanTree uses **Newtonsoft.Json** (JsonConvert.SerializeObject/DeserializeObject) for all player and clan persistence. By comparison Kits v2 and UltimateLeaderboard use **custom binary protobuf serialisation** to a byte[]. JSON is human-readable but 3–8× slower to serialise and produces 2–4× larger allocations, increasing GC pressure on every save cycle.

Impact on your server: With 100 players online and save batches running during OnServerSave, the JSON serialisation alone adds several milliseconds per frame per batch. Combined with CT-02 (main-thread writes), this amplifies every stall.

Fix (preferred): Migrate player data to the same .pb binary format used by Kits v2 — per-player files serialised with ProtocolParser. **Fix (interim):** At minimum, move serialisation off the hot path by pre-serialising to byte[] on a background thread via the worker queue (CT-02 fix).

CT-06

HIGH

IsClanAuthed() deserialises JObject on every skill purchase

IsClanAuthed() (line 2827) calls Clans.Call<JObject>("GetClan", clan) on every skill upgrade button press. This crosses the Oxide plugin boundary, executes a Clans.GetClan handler, and deserialises a JObject — allocating multiple heap objects per button press. On a busy server where clan officers rapidly upgrade buffs during an event, this contributes measurable GC pressure.

```
// Line 2831 - per-upgrade JObject deserialisation
JObject jsonObj = Clans.Call<JObject>("GetClan", clan);
var ownerid = jsonObj["owner"]?.Value<ulong>() ?? 0;
var moderators = jsonObj["moderators"]?.ToObject<List<ulong>>(); // heap alloc
```

Fix: Cache clan role data in a local dictionary keyed by clan tag, populated on OnClanMemberJoined / OnClanCreated and invalidated on role changes. Alternatively, call a specific IsClanModerator API that returns a bool instead of a full JObject if the Clans plugin supports it.

CT-07**HIGH****Full CUI rebuild + CuiHelper.AddUi() on every menu open**

Every UI menu open, tree switch, and skill click calls new CuiElementContainer() and CuiHelper.AddUi(player, container). CuiHelper.AddUi calls JsonConvert.SerializeObject on the full container before sending the RPC packet. This is the slow path — it re-serialises the entire UI on every user interaction.

```
// Line 2225, 2235, 2395, 2493, 2499, 2894, 2954 – repeated pattern
var container = SendMainMenu(player, true);
CuiHelper.AddUi(player, container); // ← JsonConvert.SerializeObject every call

// SendMainMenu builds everything from scratch each time:
// new CuiElementContainer(), loops all config.PersonalBufs, etc.
```

The established high-performance pattern (used in PulseDailyGems and UltimateLeaderboard) is: **(1)** build the static skeleton once at startup into a byte[] JsonUtf8, **(2)** embed named markers for dynamic values, **(3)** use ZString.CreateUtf8StringBuilder() to splice dynamic values into the pre-baked bytes, and **(4)** send via raw Network.Net.sv.StartWrite() RPC — bypassing CuiHelper entirely.

Interim fix (low effort): Cache the clan/personal background container JSON per-player and only rebuild on perk level changes. Do not rebuild the entire container on every tree-switch — send only the diff (the partial container approach already used in TryLevelSkill is a step in the right direction but still uses CuiHelper.AddUi).

CT-08**HIGH****GetClanMembers() returns pooled List — Pool leak risk at every call site**

GetClanMembers() (line 572) returns a Pool.Get<List<ulong>>() pooled list. Every call site that forgets to call Pool.FreeUnmanaged(ref list) leaks that list from the pool permanently, degrading pool efficiency over time. The OnClanCreated handler (line 1972) correctly uses try/finally to free, but any future caller added without the pattern will silently leak.

```
// Line 572 - returns pooled list, caller MUST free
List<ulong> GetClanMembers(string tag)
{
    var members = Pool.Get<List<ulong>>(); // ← caller owns lifecycle
    ...
    return members;
}

// Safe usage pattern (OnClanCreated, line 1972)
var members = GetClanMembers(tag);
try { foreach (var id in members) ... }
finally { Pool.FreeUnmanaged(ref members); }
```

Fix: Return an IReadOnlyList<ulong> from a cached internal structure rather than a pooled list, eliminating the lifetime management burden. Alternatively, accept an Action<IEnumerable<ulong>> callback to guarantee scope safety.

Medium Severity Findings

CT-09 MEDIUM ShouldRemoveClanData() O(n) scan of all online players on disconnect

ShouldRemoveClanData() (line 1001) iterates the entire `_BuffManager.Buffs` dictionary on every `OnPlayerDisconnected` event to check whether any remaining online player belongs to the departing player's clan. With 100 online players this is $O(n)$ on every disconnect.

```
// Line 1001 - O(n) on every disconnect
bool ShouldRemoveClanData(string tag)
{
    foreach (var kv in _BuffManager.Buffs) // iterates ALL online players
    {
        if (kv.Value.Clan == tag) return false;
    }
    return true;
}
```

Fix: Maintain a `Dictionary<string, HashSet<ulong>>` `_clanOnline` that is updated on connect/disconnect. `ShouldRemoveClanData` becomes an $O(1)$ `_clanOnline[tag].Count == 0` check.

CT-10 MEDIUM TOCTOU race in currency: balance check then separate withdraw

Currency deductions use a two-step pattern: check balance (`CanAffordGemCost`) then withdraw (`TakeGems`). These are two separate `Interface.Oxide.CallHook` calls with no atomicity guarantee. A malicious or laggy client could trigger two simultaneous upgrade presses, both passing the balance check before either deduction occurs (TOCTOU race).

```
// Lines 2928-2934 - TryLevelSkill: check then take (not atomic)
if (!CanAffordGemCost(player.userID, nextLevelData.gemCost)) { ... return; }
if (!TakeGems(player.userID, nextLevelData.gemCost)) return;
// Between these two calls a second concurrent request can pass the first check
```

Fix: Introduce a per-player `HashSet<ulong>` `_purchaseInProgress`. Guard `TryLevelSkill` / `TryLevelClanSkill` with an early exit if the player's ID is already in the set, and remove on completion. Alternatively, request an atomic `TryDeduct` API from the currency plugin developer.

CT-11 MEDIUM Instance bool flags `IsInstantGathering` / `Deforesting` — implicit shared state

Two instance-level boolean fields — `IsInstantGathering` and `Deforesting` (lines ~1570, ~1640) — are set inside `OnDispenserGathered` and read across method calls within the same synchronous execution path. While they are accessed only on the main thread today, their lifetime spans multiple methods without clear ownership, making them fragile state. If Oxide ever dispatches hooks concurrently (possible in future Rust updates) these will race.

Fix: Replace with local variables passed as parameters, or use a per-player `BuffDetails` field if they must span the lifetime of a gather event. Eliminate implicit shared mutable state between hook methods.

CT-12 MEDIUM FindEntitiesOfType uses shared `Vis.colBuffer` — fragile on concurrent queries

`FindEntitiesOfType<T>` (line 1787) uses the global static `Vis.colBuffer` collision buffer directly. This buffer is shared across all Unity physics queries on the main thread. If any other plugin (or Rust itself) calls a physics query concurrently or if this method is ever called from a non-main-thread context, the buffer contents will be corrupted, producing missed or phantom entity hits.

```
// Line 1787 - shared static buffer
int count = Physics.OverlapSphereNonAlloc(pos, radius, Vis.colBuffer, layers);
for (int i = 0; i < count; i++) { ... }
// Must clear after use - already done at line 1793 (good)
// Risk: interleaved physics queries from other plugins
```

Fix: Allocate a local `Collider[]` buffer inside the method for safety, or ensure this method is only called serially from main-thread hooks (document the constraint). The existing null-clear after use (line 1793) is correct — keep it.

CT-13**MEDIUM****SubscriptionInfo.subscribers uses List — O(n) Remove on each disconnect**

SubscriptionInfo.subscribers (line 1901) is a List<ulong>. RemoveSubscriber(id) calls subscribers.Remove(id) which is an O(n) linear scan. With a large number of players holding a given buff (e.g. WoodcuttingYield active for all online players), adding and removing on every connect/disconnect becomes measurably slow.

```
// Line 1913 - O(n) linear scan on every disconnect
public void RemoveSubscriber(ulong id)
{
    subscribers.Remove(id); // List.Remove = O(n)
    if (subscribers.Count == 0) { Unsubscribe(...); }
}
```

Fix: Replace List<ulong> with HashSet<ulong>. HashSet.Remove is O(1) amortised. Update AddSubscriber to use subscribers.Add(id) — no other changes required.

Architectural Strengths

Subscription system	Dynamic hook subscribe/unsubscribe based on active buff holders correctly avoids Oxide overhead for hooks with zero subscribers.
Pool usage in UI	SendActiveBufs uses Pool.Get>() with try/finally free — correct pattern.
Image CRC cache	TryCacheImg correctly checks FileStorage before re-downloading, stores CRC in SettingsData, and sets ReqSave flag rather than writing immediately.
Partial UI updates	TryLevelSkill sends only diff panels (SendNodeLevel, SendSkillButton, SendActiveBufs) rather than the full container after a purchase — reducing packet size per click.
HarmonyLib patching	SwitchAmmoTypesIfNeeded_Patch and OnMedicalToolAppliedPatch correctly guard IsSteamId() before buffer lookups.
Clan event hooks	OnClanMemberJoined/Gone/Disbanded/Created correctly maintain ClanBuffValues in BuffDetails, keeping in-memory buff state consistent.

Priority Fix Roadmap

#	ID	Action	Effort	Impact
1	CT-04	Fix Directory.Delete → File.Delete in RemoveOldClans()	5 min	Bug fix
2	CT-10	Add per-player purchase lock (HashSet) to prevent TOCTOU	1 hr	Correctness
3	CT-13	Change SubscriptionInfo.subscribers to HashSet<ulong>	15 min	Medium perf
4	CT-09	Add _clanOnline dictionary for O(1) ShouldRemoveClanData	1 hr	Medium perf
5	CT-03	Route SaveClanData through dirty flag + save queue	2 hrs	Critical perf
6	CT-02	Implement PlayerSaveWorker thread for all player/clan writes	4 hrs	Critical perf
7	CT-01	Preload player data at startup on background thread	4 hrs	Critical perf
8	CT-06	Cache clan auth data locally; eliminate per-click JObject	2 hrs	High perf
9	CT-08	Remove pooled-list return from GetClanMembers	1 hr	Safety
10	CT-07	Cache static UI skeleton as byte[]; use raw RPC send	8+ hrs	High perf
11	CT-05	Migrate to binary protobuf serialisation (wipe boundary)	8+ hrs	High perf

Comparison: ClanTree vs Kits v2 / UltimateLeaderboard

Feature	ClanTree	Kits v2	UltimateLeaderboard
Serialisation	JSON (Newtonsoft)	Binary protobuf	Binary protobuf
Load on connect	Sync ReadAllText	Async worker	Sync (partially fixed)
Save on event	Sync WriteAllText	Dirty + worker queue	Dirty + worker queue
Background I/O	None	Dedicated Thread	Dedicated Thread
Save jitter	No	Random 5-25s delay	Random delay
UI send method	CuiHelper.AddUi	Raw RPC packet	Raw RPC packet
UI caching	None — full rebuild	byte[] JsonUtf8 + markers	byte[] JsonUtf8 + markers

ClanTree is structurally the weakest of the three plugins on persistence. As the central plugin interfacing with currency and clan systems, its I/O improvements should be prioritised before the next high-population raid event.